



15

AUDIO ML ON PI



In the past decade, *machine learning (ML)* has taken the world by storm. It's everywhere from facial recognition to predictive text to self-driving cars, and we keep hearing about novel applications of ML seemingly every day. In this chapter, you'll use Python and TensorFlow to develop an ML-based speech recognition system that will run on an inexpensive Raspberry Pi computer.

Speech recognition systems are already deployed in a huge number of devices and appliances in the form of voice assistants such as Alexa, Google, and Siri. These systems can perform tasks ranging from setting reminders to switching on your home lights from your office. But all of these platforms require your device to be connected to the internet and for you to sign up for their services. This brings up issues of privacy, security, and power consumption. Does your light bulb *really* need to be connected to the internet to respond to a voice command? The answer is *no*. With this project, you'll get a sense of how to design a speech recognition system that works on a low-power device, without the device needing to be connected to the internet.

Some of the concepts you'll learn about through this project are:

- Using a machine learning workflow to solve a problem
- Creating an ML model with TensorFlow and Google Colab
- Streamlining an ML model for use on a Raspberry Pi
- Processing audio and generating spectrograms with the short-time Fourier transform (STFT)
- Leveraging multiprocessing to run tasks in parallel

A Machine Learning Overview

It's impossible to do justice to a topic as vast as machine learning in a single section of a single book chapter. Instead, our approach will be to treat ML as just another tool for solving a problem—in this case, how to distinguish between different spoken words. In truth, ML frameworks like TensorFlow have become so mature and easy to use these days that it's possible to effectively apply ML to a problem without being an expert in the subject. So in this section, we'll only briefly touch upon the ML terminology relevant to the project.

ML is a small part of the larger computer science discipline of *artificial intelligence (AI)*, although when AI is mentioned in the popular press, ML is usually what they mean. ML itself is made of various subdisciplines that involve different approaches and algorithms. In this project, you'll use a subset of ML called *deep learning*, which harnesses *deep neural networks (DNNs)* to identify features and patterns in large sets of data. DNNs have their origin in *artificial neural networks (ANNs)*, which are loosely based on neurons in our brains. ANNs consists of a bunch of *nodes* with multiple inputs. Each node also has a *weight* associated with it. The output of an ANN is typically a nonlinear function of the inputs and weights. This output can be connected to the input of another ANN. When you have more than one layer of ANNs, the network becomes a deep neural network. Typically, the more layers the network has—that is, the deeper it goes—the more accurate the learning model becomes.

For this project, you'll be using a *supervised learning* process, which can be divided into two phases. First is the *training phase*, where you show the model several inputs and their expected outputs. For example, if you

were trying to build a human presence detection system to recognize whether or not there's a person in a video frame, you would use the training phase to show examples of both cases (human versus no human), with each example labeled correctly. Next is the *inference phase*, where you show new inputs and the model makes predictions about them based on what it learned during training. Continuing the example, you'd show your human presence detection system new video frames, and the model would predict whether or not there's a human in each frame. (There are also *unsupervised learning* processes, in which the ML system attempts to find patterns by itself, based on unlabeled data.)

An ML model has many numerical *parameters* that help it process data. During training, these parameters are adjusted automatically to minimize errors between the expected values and the values the model predicts. Usually a class of algorithms called *gradient descent* is used to minimize the error. In addition to the parameters of an ML model, which are adjusted during training, there are also *hyperparameters*, variables that are adjusted for the model as a whole, such as which neural network architecture to use or the size of your training batch. **Figure 15-1** shows the neural network architecture I've chosen for this project.



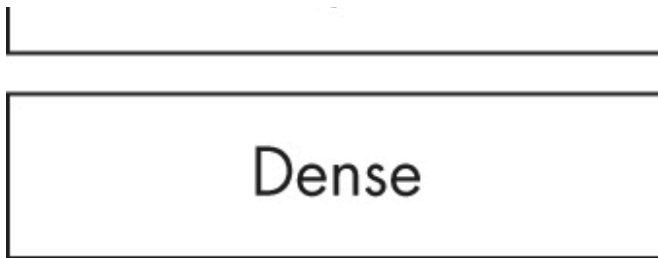


Figure 15-1: The neural network architecture for the speech recognition project

Each layer in the network architecture represents some form of processing on the data that helps improve the model's accuracy. The design of the network isn't trivial, but just defining each layer won't tell us much about how it works. A broader question to consider is *why* I've chosen this particular network. The answer is that one needs to determine the best network architecture for the project at hand via experimentation. It's common to try different neural network architectures and see which one produces the most accurate results after training. There are also architectures published by ML researchers that are known to perform well, and that's a good place to start for practical applications.

NOTE *For more information on machine learning, I highly recommend the book [Deep Learning: A Visual Approach](#) by Andrew Glassner (No Starch Press, 2021). The book will give you a good intuition about the subject without getting too much into the math or code. For a comprehensive, hands-on approach, I also recommend the online ML courses on Coursera taught by Andrew Ng.*

How It Works

In this project, you'll use Google's TensorFlow machine learning framework to train a neural network using a collection of audio files containing speech commands. Then you'll load an optimized version of the trained model onto a Raspberry Pi equipped with a microphone so the Pi can recognize the commands when you speak them. [Figure 15-2](#) shows a block diagram for the project.

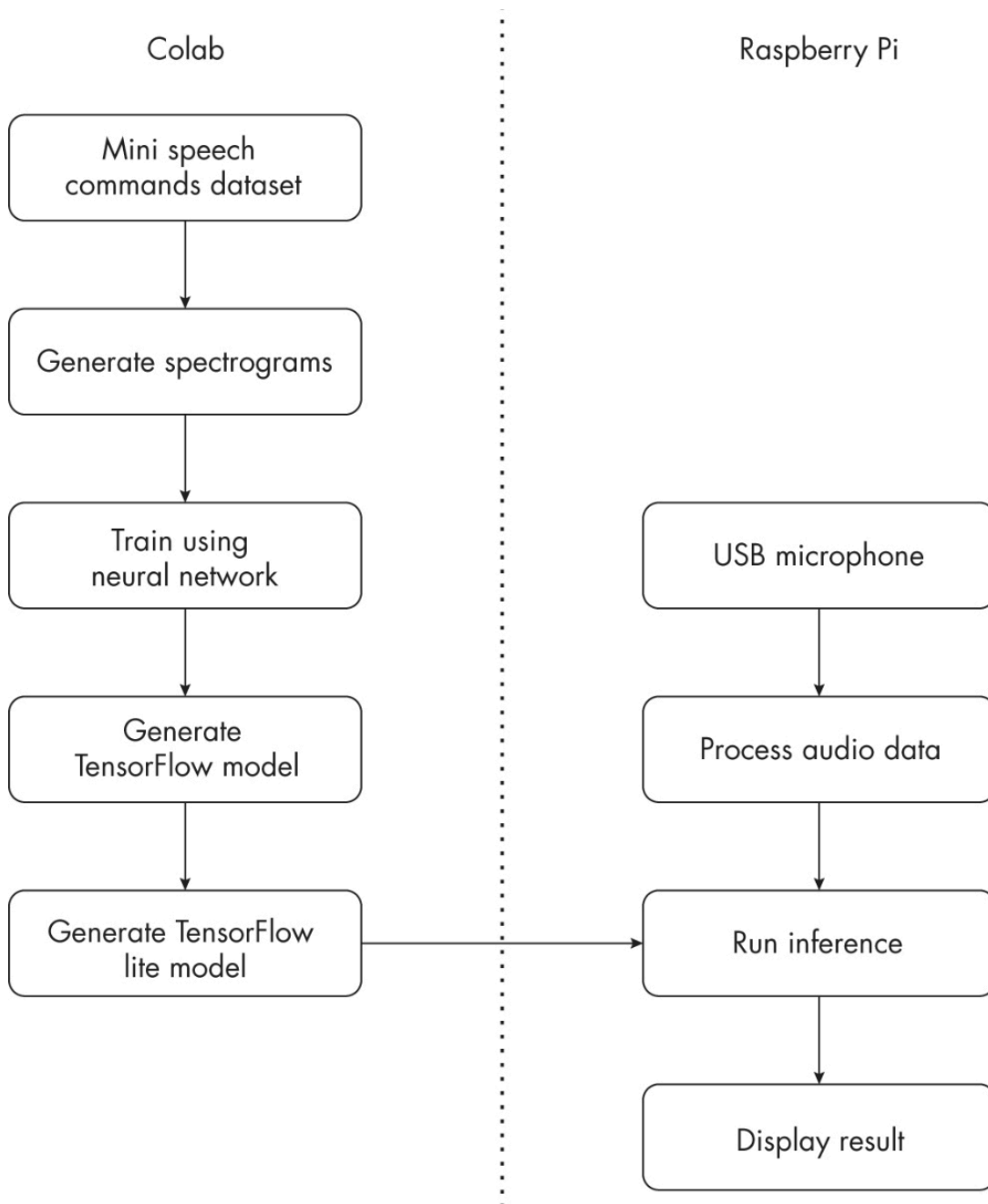


Figure 15-2: A block diagram of the speech recognition project

For the training portion of the project, you'll work in Google Colab (short for Collaboratory), a free cloud-based service that lets you write and run Python programs in your web browser. There are two advantages to using Colab. First, you don't need to install TensorFlow locally on your computer, nor deal with incompatibility issues related to various versions of TensorFlow. Second, Colab runs on machines that are likely much more powerful than yours, so the training process will go more quickly. For training data, you'll use the Mini Speech Commands dataset

from Google, a subset of a larger Speech Commands dataset published in 2018. It consists of thousands of sample recordings of the words *yes*, *no*, *up*, *down*, *left*, *right*, *stop*, and *go*, all standardized as 16-bit WAV files with a 16,000 Hz sampling rate. You'll generate a *spectrogram* of each recording, an image that shows how the frequency content of the audio changes over time, and use those spectrograms to train a deep neural network (DNN) via TensorFlow.

NOTE *The training portion of this project takes inspiration from Google's official TensorFlow example called "Simple Audio Recognition." You'll use the same neural network architecture as that example. However, the rest of the project deviates significantly from Google's example, since our goal is to recognize live audio on a Raspberry Pi, whereas the latter runs inference on existing WAV files.*

Once the training is complete, you'll convert the trained model to a simplified format called TensorFlow Lite, which is designed to run on less capable hardware such as embedded systems, and load that streamlined model onto the Raspberry Pi. There you'll run Python code to continuously monitor the audio input from a USB microphone, take spectrograms of the audio, and perform inference on that data to identify the spoken commands from the training set. You'll print out the commands that the model identifies to the serial monitor.

Spectrograms

A key step in this project is generating spectrograms of the audio data—both the preexisting data used to train the model and the real-time data encountered during inference. In **Chapter 4**, you saw how a spectral plot reveals the frequencies present in an audio sample at a particular moment in time. Then, in **Chapter 13**, you learned how spectral plots are calculated with a mathematical tool called a *discrete Fourier transform (DFT)*. A spectrogram is essentially just a series of spectral plots, generated through a sequence of Fourier transformers, which together reveal how the frequency content of some audio data evolves over time.

You need a spectrogram, rather than a single spectral plot, of each audio sample because the sound of human speech is incredibly complex. Even in the case of a single word, the frequencies present in the sound change significantly—and in distinctive ways—as the word is spoken. For this project, you'll be working with one-second-long audio clips, each consisting of 16,000 samples. If you computed a single DFT of the entire clip in one go, you wouldn't get an accurate picture of how the frequencies change over the course of the clip, and thus you wouldn't be able to reliably identify the word being spoken. Instead, you'll divide the clip into a bunch of overlapping intervals and compute the DFT for each of these intervals, giving you the series of spectral plots needed for a spectrogram.

Figure 15-3 illustrates this type of computation, called a *short-time Fourier transform (STFT)*.

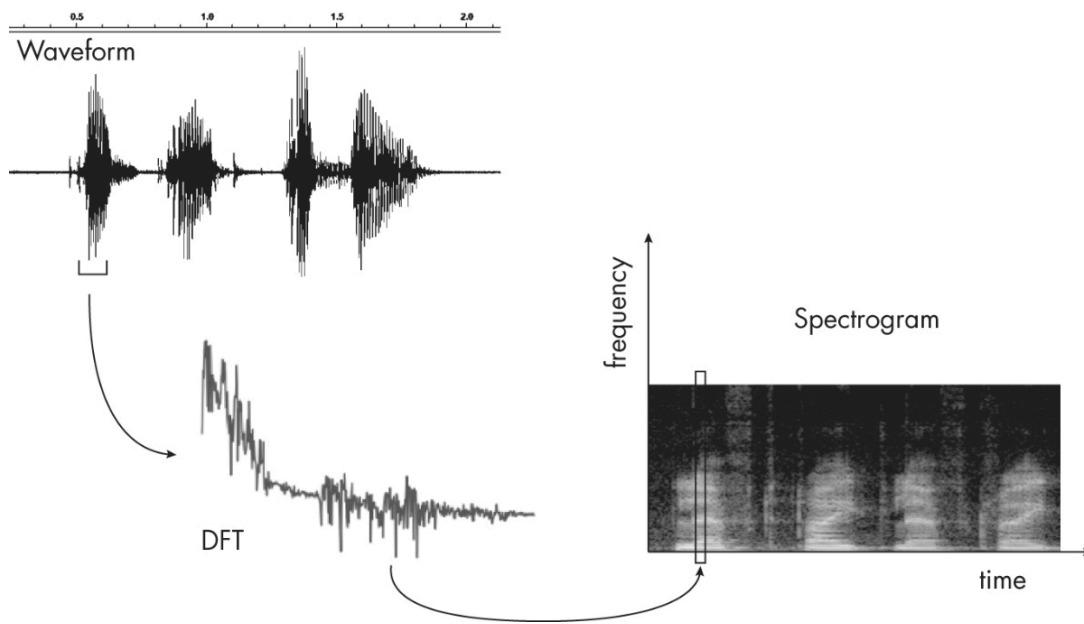


Figure 15-3: Computing the spectrogram of a signal

The STFT gives you M DFTs of the audio, taken at even time intervals. Time is shown along the x-axis of the spectrogram. Each DFT gives you N frequency bins and the intensity of the sound within each of those bins. The frequency bins are mapped to the y-axis of the spectrogram. Thus, the spectrogram takes the form of an $M \times N$ image. Each column of pixels in the image represents one of the DFTs, with color used to convey the intensity of the signal in a given frequency band.

You might be wondering why we need to use Fourier transforms at all for this project. Why not use the waveforms of the audio clips directly, instead of extracting the frequency information from those waveforms? For an answer, consider **Figure 15-4**.

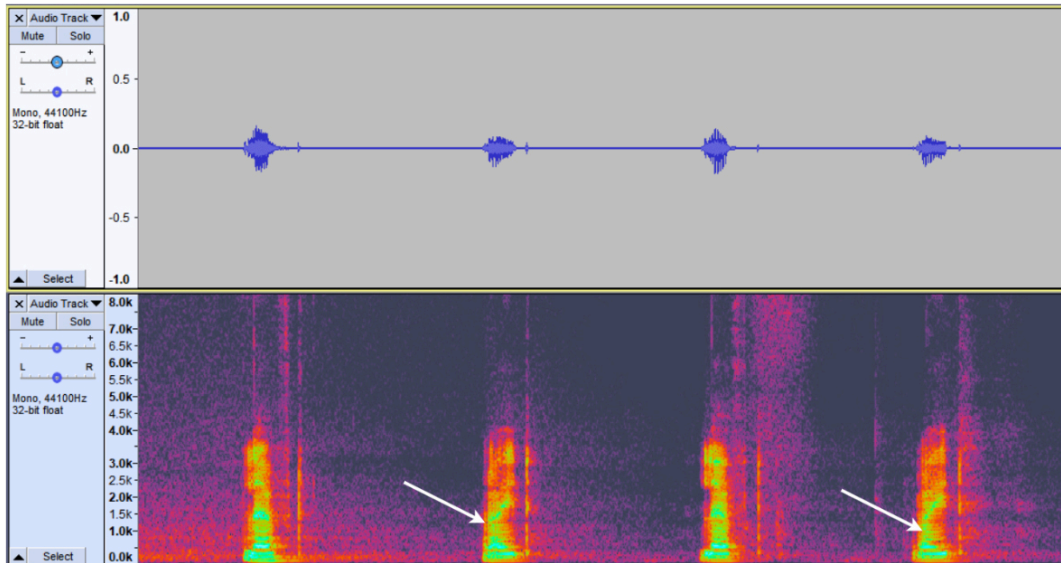


Figure 15-4: The waveform and spectrogram of speech samples

The top half of the figure shows the waveform of a recording made by speaking the sequence “Left, right, left, right.” The bottom half of the figure shows a spectrogram of that recording. Looking just at the waveform, you can see that the two *lefts* look vaguely similar, as do the two *rights*, but it’s hard to pick out strong identifying characteristics of each word’s waveform. By contrast, the spectrogram reveals more visual features associated with each word, like the bright C-shaped curve (shown by the arrows) in each instance of *right*. We can see these distinctive features more clearly with our own eyes, and a neural network can “see” them as well.

In the end, since a spectrogram is essentially an image, taking spectrograms of the data turns a speech recognition problem into an image classification problem, allowing us to leverage the rich set of ML techniques that already exist for classifying images. (Of course, a waveform can be treated as an image too, but as you’ve seen, a spectrogram is better at

capturing the “signature” of the audio data and hence more suited for ML applications.)

Inference on the Raspberry Pi

The code on the Raspberry Pi must accomplish several tasks: it needs to read the audio input from the attached microphone, compute the spectrogram of that audio, and do inference using the trained TensorFlow Lite model. Here’s one possible sequence of operations:

1. Read microphone data for one second.
2. Process data.
3. Do inference.
4. Repeat.

There’s a big problem with this approach, however. While you’re busy with steps 2 and 3, more speech data could be coming in, which you’ll end up missing. The solution is to use Python multiprocessing to perform different tasks concurrently. Your main process will just collect the audio data and put it in a queue. In a separate, simultaneous process, you’ll take this data out of the queue and run inference on it. Here’s what the new scheme looks like:

Main Process

1. Read microphone data for one second.
2. Put data into the queue.

Inference Process

1. Check if there’s any data in the queue.
2. Run inference on the data.

Now the main process won’t miss any audio input, since putting data into the queue is a very quick operation. But there’s another problem. You’re collecting one-second audio samples continuously from the micro-

phone and processing them, but you can't assume that all spoken commands will fit cleanly into those one-second intervals. A command could come at an edge and be broken up across two consecutive samples, in which case it probably won't be identified during inference. A better approach is to create overlapping samples, as follows:

Main Process

1. For the very first frame, collect a two-second sample.
2. Put the two-second sample into the queue.
3. Collect another one-second sample.
4. Create a two-second sample by moving the latter half of the sample from step 2 to the front and replacing the second half with the sample from step 3.
5. Return to step 2.

Inference Process

1. Check if there's any data in the queue.
2. Do inference on a one-second portion of the two-second data based on peak amplitude.
3. Return to step 1.

In this new scheme, each sample placed into the queue is two seconds long, but there's a one-second overlap between consecutive samples, as illustrated in [Figure 15-5](#). This way, even if a word is partially cut off in one sample, you'll get the full word in the next sample. You'll still run inference on only one-second clips, which you'll center on the point in the two-second sample that has the highest amplitude value. This is the part of the sample most likely to contain a spoken word. You need the clips to be one second long for consistency with the training data.

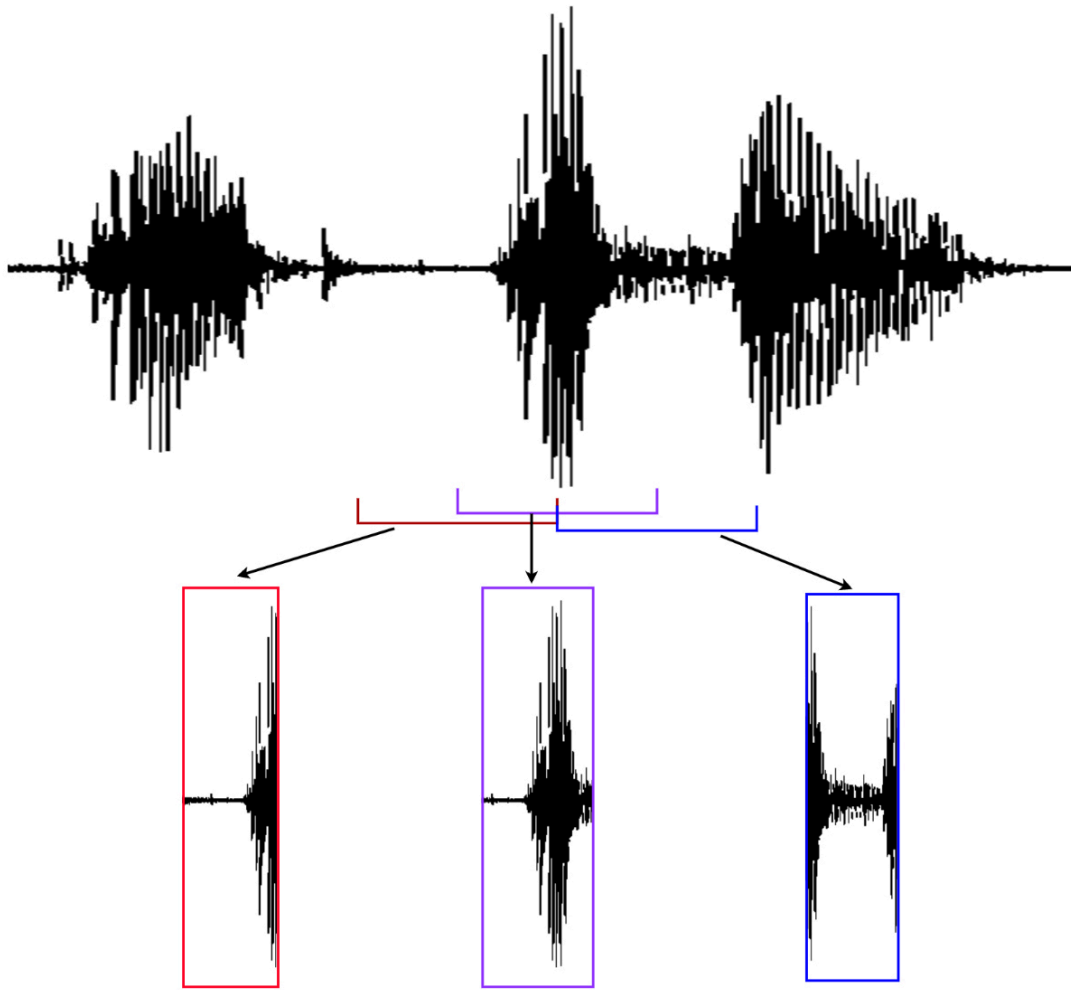


Figure 15-5: The two-frame overlapping scheme

Through this combination of multiprocessing and overlapping samples, you'll design a speech recognition system that minimizes missing inputs and improves inference results.

Requirements

For this project, you'll need to sign up with Google Colab to train your ML model. On the Raspberry Pi, you'll need the following Python modules:

- `tf.lite_runtime` for running the TensorFlow inference
- `scipy` for computing the STFT of audio waveforms
- `numpy` arrays for handling audio data
- `pyaudio` for streaming audio data from the microphone input

The installation for these modules is covered in [Appendix B](#). You'll also use Python's built-in `multiprocessing` module for running ML inference in a separate thread from the audio processing thread.

In the hardware department, you'll need the following:

- One Raspberry Pi 3B+ or newer
- One 5 V power supply for the Raspberry Pi
- One 16GB SD card
- One single-channel USB microphone compatible with the Pi

Various types of USB microphones are compatible with the Raspberry Pi. [Figure 15-6](#) shows an example.



Figure 15-6: A USB microphone for the Raspberry Pi

To check if your Pi can recognize your USB microphone, SSH into your Pi and run the following command:

```
$ dmesg -w
```

Now plug your microphone into a USB port on the Pi. You should see something similar to the following output:

```
[26965.023138] usb 1-1.3: New USB device found, idVendor=cafe,  
idProduct=4010, bcdDevice= 1.00
```

```
[26965.023163] usb 1-1.3: New USB device strings: Mfr=1, Product=2,  
SerialNumber=3
```

```
[26965.023179] usb 1-1.3: Product: Mico
```

```
[26965.023194] usb 1-1.3: Manufacturer: Electronut Labs
```

```
[26965.023209] usb 1-1.3: SerialNumber: 123456
```

The information should match the specs of your microphone, indicating it's been correctly identified.

The Code

The code for this project exists in two parts: the training portion, which you'll run in Google Colab, and the inference portion, which you'll run on your Raspberry Pi. We'll examine these parts one at a time.

Training the Model in Google Colab

In this section, we'll look at the Google Colab code needed to train the speech recognition model. I recommend working with Colab in the Chrome web browser. You'll begin by getting set up and downloading the training dataset. Then you'll run some code to get to know the data. You'll clean up the data to prepare it for training and explore how to generate spectrograms from the data. Finally, you'll put what you've learned to work by creating and training the model. The end result will be a *.tflite* file, a streamlined TensorFlow Lite version of the trained model that you can load onto your Raspberry Pi. You can also download this file

from the book's GitHub repository at

<https://github.com/mkvenkit/pp2e/tree/main/audioml>.

A Google Colab notebook consists of a series of cells where you enter one or more lines of Python code. Once you've entered your desired code into a cell, you run it by clicking the Play icon in the top-left corner of the cell. Any output associated with that cell's code will then appear beneath the cell. Throughout this section, each code listing will represent a complete Google Colab cell. The cell's output, if any, will be shown in gray at the end of the listing, beneath a dashed line.

Setting Up

You begin your Colab notebook with some initial setup. First you import the required Python modules:

```
import os
```

```
import pathlib
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import scipy
```

```
import scipy.signal
```

```
from scipy.io import wavfile
```

```
import glob
```

```
import tensorflow as tf
```

```
from tensorflow.keras.layers.experimental import preprocessing
```

```
from tensorflow.keras import layers
```



```
from tensorflow.keras import models
```

```
from tensorflow.keras import applications
```

In the next cell, you do some initialization:

```
# set seed for random functions
```

```
seed = 42
```

```
tf.random.set_seed(seed)
```

```
np.random.seed(seed)
```

Here you initialize the random functions you'll be using to shuffle the input filenames.

Next, download the training data:

```
data_dir = 'data/mini_speech_commands'
```

```
data_path = pathlib.Path(data_dir)
```

```
filename = 'mini_speech_commands.zip'
```

```
url =
```

```
"http://storage.googleapis.com/download.tensorflow.org/data/mini_speech_commands.zip"
```

```
if not data_path.exists():
```

```
    tf.keras.utils.get_file(filename, origin=url, extract=True, cache_dir='.',
```

```
                           cache_subdir='data')
```

This cell downloads the Mini Speech Commands dataset from Google and extracts the data into a directory called *data*. Since you're using Colab, the data will be downloaded to the filesystem on the cloud, not to your local machine, and when your session ends, these files will be deleted.

While the session is still active, however, you don't want to have to keep downloading the data every time you run the cell. The `tf.keras.utils.get_file()` function caches the data so you won't need to keep downloading it.

Getting to Know the Data

Before you start training your model, it would be useful to take a look at what you just downloaded to get to know your data. You can use Python's `glob` module, which helps you find files and directories through pattern matching:

```
glob.glob(data_dir + '/*')
```

```
['data/mini_speech_commands/up',
```

```
'data/mini_speech_commands/no',
```

```
'data/mini_speech_commands/README.md',
```

```
'data/mini_speech_commands/stop',
```

```
'data/mini_speech_commands/left',
```

```
'data/mini_speech_commands/right',
```

```
'data/mini_speech_commands/go',
```

```
'data/mini_speech_commands/down',
```

```
'data/mini_speech_commands/yes']
```

You pass `glob` the `'/*'` pattern to list all the first-level directories within the `data` directory (`*` is a wildcard character). The output shows you that the dataset comes with a *README.md* text file and eight subdirectories for the eight speech commands you'll be training the model to identify. For convenience, you create a list of the commands:

```
commands = ['up', 'no', 'stop', 'left', 'right', 'go', 'down', 'yes']
```

In your machine learning model, you'll be matching audio samples to a `label_id` integer denoting one of the commands. These integers will correspond to the indices from the `commands` list. For example, a `label_id` of `0` indicates `'up'`, and a `label_id` of `6` indicates `'down'`.

Now take a look at what's in those subdirectories:

```
❶ wav_file_names = glob.glob(data_dir + '/*/*')
```

```
❷ np.random.shuffle(wav_file_names)
```

```
print(len(wav_file_names))
```

```
for file_name in wav_file_names[:5]:
```

```
    print(file_name)
```

```
8000
```

```
data/mini_speech_commands/down/27c30960_nohash_0.wav
```

```
data/mini_speech_commands/go/19785c4e_nohash_0.wav
```

```
data/mini_speech_commands/yes/d9b50b8b_nohash_0.wav
```

```
data/mini_speech_commands/no/f953e1af_nohash_3.wav
```

```
data/mini_speech_commands/stop/f632210f_nohash_0.wav
```

You use `glob` again, this time showing it the pattern `'/*/*'` to list all the files in the subdirectories ❶. Then you randomly shuffle the returned list of filenames to reduce any bias in the training data ❷. You print the total number of files found, as well as the first five filenames. The output indicates that there are 8,000 WAV files in the dataset, and it gives you

some idea of how the files are named—for example, *f632210f_nohash_0.wav*.

Next, take a look at some individual WAV files from the dataset:

```
filepath = 'data/mini_speech_commands/stop/f632210f_nohash_1.wav' ❶
```

```
rate, data = wavfile.read(filepath) ❷
```

```
print("rate = {}, data.shape = {}, data.dtype = {}".format(rate, data.shape, data.dtype))
```

```
filepath = 'data/mini_speech_commands/no/f953e1af_nohash_3.wav'
```

```
rate, data = wavfile.read(filepath)
```

```
print("rate = {}, data.shape = {}, data.dtype = {}".format(rate, data.shape, data.dtype))
```

```
rate = 16000, data.shape = (13654,), data.dtype = int16
```

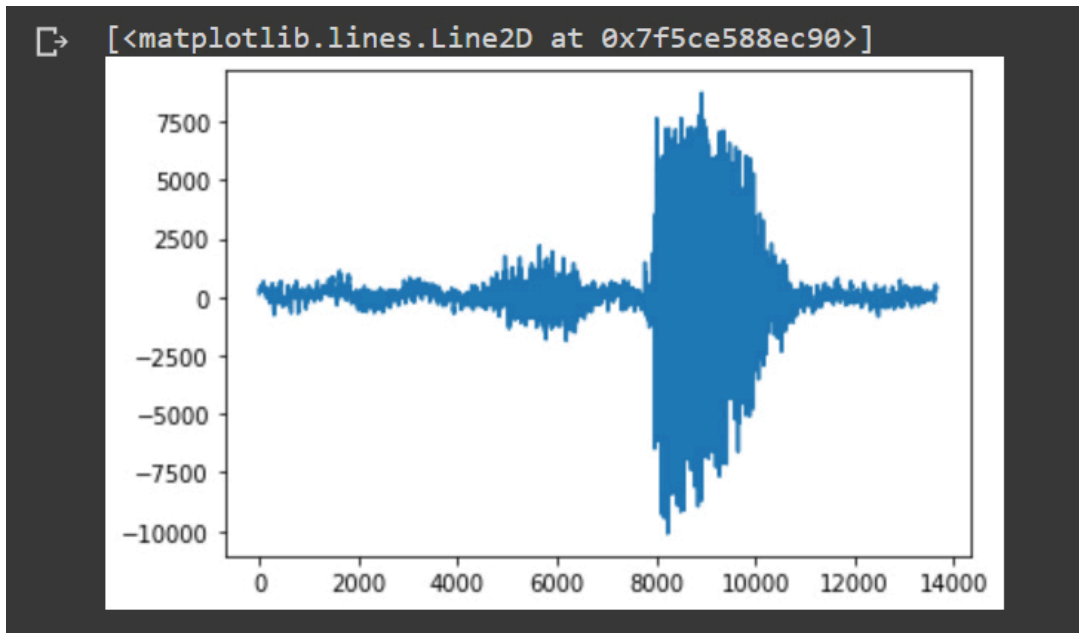
```
rate = 16000, data.shape = (16000,), data.dtype = int16
```

You set the name of a WAV file you want to look at ❶ and use the `wavfile` module from `scipy` to read data from the file ❷. Then you print the sampling rate, shape (number of samples), and type of the data. You do the same for a second WAV file. The output shows that the sampling rates of both the WAV files are 16,000, as expected, and that each sample is a 16-bit integer for both—also expected. However, the shape indicates the first file has only 13,654 samples, and this is a problem. To train the neural network, each WAV file needs to have the same length; in this case, you'd like each recording to be one second, or 16,000 samples, long. Unfortunately, not all the files in the dataset fit that standard. We'll look at a solution to this problem shortly, but first, try plotting the data from one of these WAV files:

```
filepath = 'data/mini_speech_commands/stop/f632210f_nohash_1.wav'
```

```
rate, data = wavfile.read(filepath)
```

```
❶ plt.plot(data)
```



You use `matplotlib` to create a plot of the audio waveform ❶. The WAV files in this dataset contain 16-bit signed data, which can range from $-32,768$ to $+32,767$. The y-axis of the plot shows you that the data in this file ranges from only around $-10,000$ to $+7,500$. The plot's x-axis also underscores that the data is short of the necessary 16,000 samples—the axis runs only to 14,000.

Cleaning the Data

You've seen that the dataset needs to be standardized so that each clip is one second long. This type of preparatory work is called *data cleaning*, and you can do it by padding the audio data with zeros until it reaches a length of 16,000 samples. You should clean the data further by *normalizing* it—mapping the value of each sample from range $[-32,768, +32,767]$ to range $[-1, 1]$. This type of normalization is crucial for machine learning, as keeping the input data small and uniform will help the training. (For the mathematically curious, large numbers in the inputs will cause

problems in the convergence of gradient descent algorithms used to train the data.)

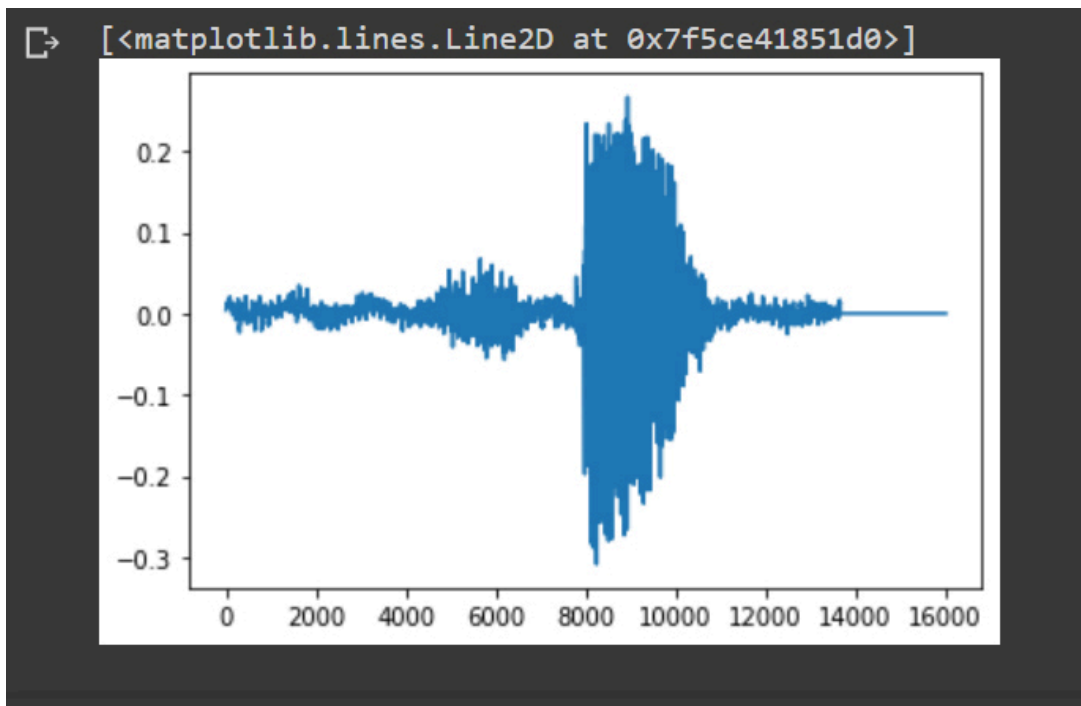
As an example of data cleaning, here you apply both padding and normalization to the WAV file you viewed in the previous listing. Then you plot the results to confirm that the cleaning has worked.

```
❶ padded_data = np.zeros((16000,), dtype=np.int16)

❷ padded_data[:data.shape[0]] = data

❸ norm_data = np.array(padded_data/32768.0, dtype=np.float32)

plt.plot(norm_data)
```



You create a 16-bit `numpy` array of length 16,000 filled with zeros ❶. Then you use the array slice operator, `[:]`, to copy the contents of the too-short WAV file into the beginning of the array ❷. Here `data.shape[0]` gives you the number of samples in the original WAV file, since `data.shape` is a tuple in the form `(13654,)`. You now have one second of WAV data, consisting of the original audio data followed by a padding

of zeros as needed. You next create a normalized version of the data by dividing the values in the array by 32,768, the maximum value a 16-bit integer could have ❸. Then you plot the data.

The x-axis of the output shows that the data has been padded to extend to 16,000 samples, with the values from around 14,000 to 16,000 all being zero. Also, the y-axis shows that the values have all been normalized to fall nicely within the range of $(-1, 1)$.

Looking at Spectrograms

As we've discussed, you won't be training your model on the raw data from the WAV files. Instead, you'll generate spectrograms of the files and use them to train the model. Here's an example of how to generate a spectrogram:

```
filepath = 'data/mini_speech_commands/yes/00f0204f_nohash_0.wav'
```

```
rate, data = wavfile.read(filepath)
```

```
❶ f, t, spec = scipy.signal.stft(data, fs=16000, nperseg=255,
```

```
    noverlap = 124, nfft=256)
```

```
❷ spec = np.abs(spec)
```

```
print("spec: min = {}, max = {}, shape = {}, dtype =  
{}").format(np.min(spec),
```

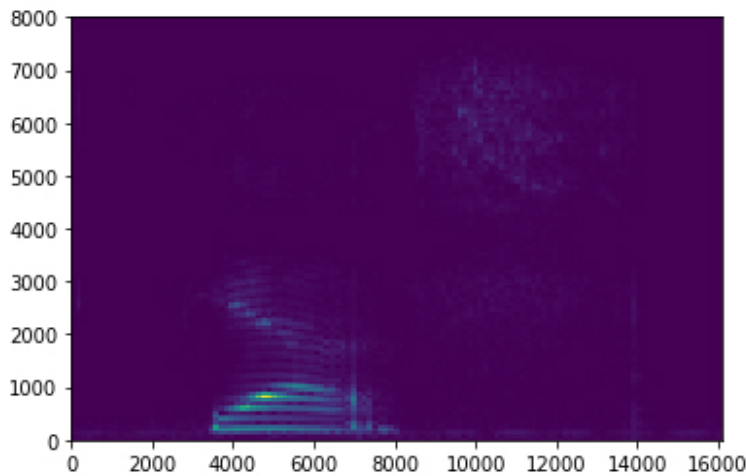
```
            np.max(spec), spec.shape, spec.dtype))
```

```
❸ X = t * 129*124
```

```
❹ plt.pcolormesh(X, f, spec)
```

```
spec: min = 0.0, max = 2089.085693359375, shape = (129, 124), dtype =
```

float32



You pick an arbitrary WAV file from the `yes` subdirectory and extract its data using the `wavfile` module from `scipy`, as before. Then you use the `scipy.signal.stft()` function to compute the spectrogram of the data ❶. In this function call, `fs` is the sampling rate, `nperseg` is the length of each segment, and `noverlap` is the number of overlapping samples between consecutive segments. The `stft()` function returns a tuple comprising three members: `f`, an array of frequencies; `t`, an array of the time intervals mapped to the range `[0.0, 1.0]`; and `spec`, the STFT itself, a grid of 129×124 complex numbers (these dimensions are given as `shape` in the output). You use `np.abs()` to convert the complex numbers in `spec` into real numbers ❷. Then you print some information about the computed spectrogram. Next, you create an array `X` to hold the sample numbers corresponding to the time intervals ❸. You get these by multiplying `t` by the dimensions of the grid. Finally, you use the `pcolormesh()` method to plot the grid in `spec`, using the values in `X` as the grid's x-axis and the values in `f` as the y-axis ❹.

The output shows the spectrogram. This 129×124 grid of values (an image), and many more like it, will be the input for the neural network. The bright spots around 1,000 Hz and lower, starting around 4,000 samples in, are where the frequency content is most prominent, while darker areas represent less prominent frequencies.

NOTE Notice that the y-axis in the spectrogram images goes up to only about 8,000 Hz. This is a consequence of the sampling theorem in digital signal processing, which states that the maximum frequency that can be accurately measured in a digitally sampled signal is half the sampling rate. In this case, that maximum frequency works out to $16,000/2 = 8,000$ Hz.

Training the Model

You're now ready to turn your attention to training the ML model, and that largely means leaving behind Python libraries like `numpy` and `scipy` in favor of TensorFlow methods and data structures like `tf.Tensor` and `tf.data.Dataset`. You've been using `numpy` and `scipy` so far because they've provided a convenient way to explore the speech commands dataset, and in fact you could continue using them, but then you'd miss out on the optimization opportunities provided by TensorFlow, which is designed for large-scale ML systems. You'll find that TensorFlow has near-identical functions for most of the computations you've done until now, so the transition will be smooth. For our purposes, when I refer to a *tensor* in the upcoming discussion, understand that it's similar to talking about a `numpy` array.

To train the ML model, you need to be able to extract the spectrogram and label ID (the spoken command) from the filepath of an input audio file. For that, first create a function that computes an STFT:

```
def stft(x):
```

```
    ❶ f, t, spec = scipy.signal.stft(x.numpy(), fs=16000, nperseg=255,
```

```
        noverlap=124, nfft=256)
```

```
    ❷ return tf.convert_to_tensor(np.abs(spec))
```

The function takes in `x`, the data extracted from a WAV file, and computes its STFT using `scipy`, as before ❶. Then you convert the returned `numpy` array to a `tf.Tensor` object and return the result ❷. There is, in fact, a TensorFlow method called `tf.signal.stft()` that's similar to the

`scipy.signal.stft()` method, so why not use it? The answer is that the TensorFlow method won't be available on the Raspberry Pi, where you'll be using the slimmed-down TensorFlow Lite interpreter. Any preprocessing you do during the training phase should be identical to any preprocessing you do during inference, so you need to ensure that you use the same functions in Colab as you'll use on the Pi.

Now you can make use of your `stft()` function in a helper function that extracts the spectrogram and label ID from a filepath.

```
def get_spec_label_pair(filepath):  
  
    # read WAV file  
  
    file_data = tf.io.read_file(filepath)  
  
    data, rate = tf.audio.decode_wav(file_data)  
  
    data = tf.squeeze(data, axis=-1)  
  
    # add zero padding for N < 16000  
  
    ❶ zero_padding = tf.zeros([16000] - tf.shape(data), dtype=tf.float32)  
  
    # combine data with zero padding  
  
    ❷ padded_data = tf.concat([data, zero_padding], 0)  
  
    # compute spectrogram  
  
    ❸ spec = tf.py_function(func=stft, inp=[padded_data], Tout=tf.float32)  
  
    spec.set_shape((129, 124))  
  
    spec = tf.expand_dims(spec, -1)  
  
    # get label
```

```

❷ cmd = tf.strings.split(filepath, os.path.sep)[-2]

❸ label_id = tf.argmax(tf.cast(cmd == commands, "uint32"))

```

```

# return tuple

```

```

return (spec, label_id)

```

You start by reading the file using `tf.io.read_file()` and decoding the WAV format using the `tf.audio.decode_wav()` function. (The latter is comparable to the `scipy.io.wavfile.read()` function you used previously.) You then use `tf.squeeze()` to change the shape of the `data` tensor from $(N, 1)$ to $(N,)$, which is required for functions coming ahead. Next, you create a tensor for zero-padding the data ❶. Tensors are immutable objects, however, so you can't copy the WAV data directly into a tensor full of zeros, as you did earlier with `numpy` arrays. Instead, you create a tensor with the exact number of zeros you need to pad the data, and then you concatenate it with the data tensor ❷.

You next use `tf.py_function()` to call the `stft()` function you defined earlier ❸. In this call, you also need to specify the input and the data type of the output. This is a common method for calling a non-TensorFlow function from TensorFlow. You then do some reshaping of the tensor returned by `stft()`. First you use `set_shape()` to reshape it to $(129, 124)$, which is necessary because you're going from a TensorFlow function to a Python function and back. Then you run `tf.expand_dims(spec, -1)` to add a third dimension, going from $(129, 124)$ to $(129, 124, 1)$. The extra dimension is needed for the neural network model you'll be building. Finally, you extract the label (for example, `'no'`) associated with the filepath ❹ and convert the label string to the integer `label_id` ❺, which is the index of the string in your `commands` list.

Next, you need to get the input files ready for training. Recall that you had 8,000 audio files in the subdirectories and that you randomly shuffled their filepath strings and put them into a list called `wav_file_names`. Now you'll partition the data into three: 80 percent, or

6,400 files, for training; 10 percent, or 800 files, for validation; and the other 10 percent for testing. Such partitioning is a common practice in machine learning. Once the model is trained using training data, you can use the validation data to tweak the model's accuracy by changing the hyperparameters. The testing data is used only for checking the final accuracy of the (tweaked) model.

```
train_files = wav_file_names[:6400]
```

```
val_files = wav_file_names[6400:7200]
```

```
test_files = wav_file_names[7200:]
```

Now you load the filepath strings into TensorFlow `Dataset` objects. These objects are critical to working with TensorFlow; they hold your input data and allow for data transformations, and all this can happen at a large scale:

```
train_ds = tf.data.Dataset.from_tensor_slices(train_files)
```

```
val_ds = tf.data.Dataset.from_tensor_slices(val_files)
```

```
test_ds = tf.data.Dataset.from_tensor_slices(test_files)
```

Take a look at what you just created:

```
for val in train_ds.take(5):
```

```
    print(val)
```

```
-----  
tf.Tensor(b'data/mini_speech_commands/stop/b4aa9fef_nohash_2.wav',  
shape=(), dtype=string)
```

```
tf.Tensor(b'data/mini_speech_commands/stop/962f27eb_nohash_0.wav',  
shape=(), dtype=string)
```

```
--snip--
```

```
tf.Tensor(b'data/mini_speech_commands/left/cf87b736_nohash_1.wav',  
shape=(), dtype=string)
```

Each `Dataset` object contains a bunch of tensors of type `string`, each holding a filepath. What you really need, however, are the `(spec, label_id)` pairs corresponding to those filepaths. You create those here:

```
train_ds = train_ds.map(get_spec_label_pair)
```

```
val_ds = val_ds.map(get_spec_label_pair)
```

```
test_ds = test_ds.map(get_spec_label_pair)
```

You use `map()` to apply your `get_spec_label_pair()` function to each `Dataset` object. This technique of mapping a function to a list of things is common in computing. Essentially, you're going through each filepath in the `Dataset` object, calling `get_spec_label_pair()` on it, and storing the resulting `(spec, label_id)` pair in a new `Dataset` object.

Now you further prepare the dataset for training by splitting it up into smaller batches:

```
batch_size = 64
```

```
train_ds = train_ds.batch(batch_size)
```

```
val_ds = val_ds.batch(batch_size)
```

Here you set the training and validation datasets to have a batch size of 64. This is a common technique for speeding up the training process. If you tried to work with all 6,400 training samples and 800 validation samples at once, it would require a huge amount of memory and would slow down the training.

Now you're finally ready to create your neural network model:

```
❶ input_shape = (129, 124, 1)
```

❷ `num_labels = 8`

`norm_layer = preprocessing.Normalization()`

❸ `norm_layer.adapt(train_ds.map(lambda x, _: x))`

❹ `model = models.Sequential([`

`layers.Input(shape=input_shape),`

`preprocessing.Resizing(32, 32),`

`norm_layer,`

`layers.Conv2D(32, 3, activation='relu'),`

`layers.Conv2D(64, 3, activation='relu'),`

`layers.MaxPooling2D(),`

`layers.Dropout(0.25),`

`layers.Flatten(),`

`layers.Dense(128, activation='relu'),`

`layers.Dropout(0.5),`

`layers.Dense(num_labels),`

`])`

❺ `model.summary()`

Model: "sequential_3"

Layer (type)	Output Shape	Param #
=====		
resizing_3 (Resizing)	(None, 32, 32, 1)	0
normalization_3 (Normalization)	(None, 32, 32, 1)	3
conv2d_5 (Conv2D)	(None, 30, 30, 32)	320
conv2d_6 (Conv2D)	(None, 28, 28, 64)	18496
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 64)	0
dropout_6 (Dropout)	(None, 14, 14, 64)	0
flatten_3 (Flatten)	(None, 12544)	0
dense_6 (Dense)	(None, 128)	1605760
dropout_7 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 8)	1032
=====		

Total params: 1,625,611

Trainable params: 1,625,608

Non-trainable params: 3

You set the shape of the input into the first layer of the model ❶ and then set the number of labels ❷, which will be the number of units in the model’s output layer. Next, you set up a normalization layer for the spectrogram data. This will scale and shift the data to a distribution centered on 1 with a standard deviation of 1. This is a common practice in ML that improves training. Don’t let the `lambda` scare you ❸. All it’s doing is

defining an anonymous function that picks out just the spectrogram from each `(spec, label_id)` pair in the training dataset. The `x, _: x` is just saying to ignore the second element in the pair and return only the first element.

You next create the neural network model, one layer at a time ④. The layers correspond to the architecture we viewed earlier in [Figure 15-1](#). Finally, you print out a summary of the model ⑤, which is shown in the output. The summary tells you all the layers in the model, the shape of the output tensor at each stage, and the number of trainable parameters in each layer.

Now you need to compile the model. The compilation step sets the optimizer, the loss function, and the data collection metrics for the model:

```
model.compile(

    optimizer=tf.keras.optimizers.Adam(),

    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),

    metrics=['accuracy'],

)
```

A *loss function* is a function used to measure how well a neural network is doing by comparing the output of the model to the known correct training data. An *optimizer* is a method used to adjust the trainable parameters in a model to reduce the losses. In this case, you're using an optimizer of type `Adam` and a loss function of type `SparseCategoricalCrossentropy`. You also get set up to collect some accuracy metrics, which you'll use to check how the training went.

Next, you train the model:

```
EPOCHS = 10
```



```

history = model.fit(

    train_ds,

    validation_data=val_ds,

    epochs=EPOCHS,

    callbacks=tf.keras.callbacks.EarlyStopping(verbose=1, patience=15), ❶

)

```

Epoch 1/10

```

100/100 [=====] - 38s 371ms/step - loss:
1.7219 - accuracy: 0.3700

- val_loss: 1.2672 - val_accuracy: 0.5763

```

Epoch 2/10

```

100/100 [=====] - 37s 368ms/step - loss:
1.1791 - accuracy: 0.5756

- val_loss: 0.9616 - val_accuracy: 0.6650

```

--snip--

Epoch 10/10

```

100/100 [=====] - 39s 388ms/step - loss:
0.3897 - accuracy: 0.8639

- val_loss: 0.4766 - val_accuracy: 0.8450

```

You train the model by passing the training dataset `train_ds` into `model.fit()`. You also specify the validation dataset `val_ds`, which is used to evaluate how accurate the model is. The training takes place over

10 *epochs*. During each epoch, the complete set of training data is shown to the neural network. The data is randomly shuffled each time, so training across multiple epochs allows the model to learn better. You use the `callback` option ❶ to set up a function to exit the training if it turns out that the training loss is no longer decreasing with each epoch.

Running this Colab cell will take some time. The progress will be shown on the screen as the training is in process. Looking at the output, the `val_accuracy` listed under Epoch 10 shows that the model was about 85 percent accurate at running inference on the validation data by the end of the training. (The `val_accuracy` metric corresponds to the validation data, while `accuracy` corresponds to the training data.)

Now you can try the model by running inference on the testing portion of the data:

```
test_audio = []
```

```
test_labels = []
```

❶ for audio, label in test_ds:

```
    test_audio.append(audio.numpy())
```

```
    test_labels.append(label.numpy())
```

❷ `test_audio = np.array(test_audio)`

```
test_labels = np.array(test_labels)
```

❸ `y_pred = np.argmax(model.predict(test_audio), axis=1)`

```
y_true = test_labels
```

❹ `test_acc = sum(y_pred == y_true) / len(y_true)`

```
print(f'Test set accuracy: {test_acc:.0%}')
```

25/25 [=====] - 1s 35ms/step

Test set accuracy: 84%

You first fill in two lists, `test_audio` and `test_labels`, by iterating through the test dataset `test_ds` ❶. Then you create `numpy` arrays from these lists ❷ and run inference on the data ❸. You compute the test accuracy by summing up the number of times the predictions matched the true value and dividing them by the total number of items ❹. The output shows an accuracy of 84 percent. Not perfect, but good enough for this project.

Exporting the Model to the Pi

Congratulations! You have a fully trained machine learning model. Now you need to get it from Colab onto your Raspberry Pi. The first step is to save it:

```
model.save('audioml.sav')
```

This saves the model to a file on the cloud called *audioml.sav*. Next, convert that file to the TensorFlow Lite format so you can use it on your Pi:

```
❶ converter = tf.lite.TFLiteConverter.from_saved_model('audioml.sav')
```

```
❷ tflite_model = converter.convert()
```

```
❸ with open('audioml.tflite', 'wb') as f:
```

```
    f.write(tflite_model)
```

You create a `TFLiteConverter` object, passing in the saved model filename ❶. Then you do the conversion ❷ and write the simplified TensorFlow model to a file called *audioml.tflite* ❸. Now you need to download this *.tflite* file from Colab onto your computer. Running the following snippet will give you a browser prompt to save the *.tflite* file:

```
from google.colab import files
```

```
files.download('audioml.tflite')
```

Once you have the file, you can transfer it to your Raspberry Pi using SSH as we've discussed in other chapters.

Using the Model on the Raspberry Pi

Now we'll turn our attention to the Raspberry Pi portion of the code. This code uses parallel processing to take in audio data from the microphone, prepare that data for your trained ML model, and show the data to the model to perform inference. As usual, you can write the code on your local machine and then transfer it to your Pi via SSH. To view the complete code, see **"The Complete Code"** on **page 389**. You can also download the code from **<https://github.com/mkvenkit/pp2e/tree/main/audioml>**.

Setting Up

Start by importing the required modules:

```
from scipy.io import wavfile
```

```
from scipy import signal
```

```
import numpy as np
```

```
import argparse
```

```
import pyaudio
```

```
import wave
```

```
import time
```

```
from tflite_runtime.interpreter import Interpreter
```

```
from multiprocessing import Process, Queue
```

Next, you initialize some parameters that are defined as global variables:

```
VERBOSE_DEBUG = False
```

```
CHUNK = 4000
```

```
FORMAT = pyaudio.paInt16
```

```
CHANNELS = 1
```

```
SAMPLE_RATE = 16000
```

```
RECORD_SECONDS = 1
```

```
NCHUNKS = int((SAMPLE_RATE * RECORD_SECONDS) / CHUNK)
```

```
ND = 2 * CHUNK * NCHUNKS
```

```
NDH = ND // 2
```

```
# device index of microphone
```

```
❶ dev_index = -1
```

`VERBOSE_DEBUG` is a flag you'll use in many places in the code. For now, you set it to `False`, but if set to `True` (via a command line option), it will print out a lot of debugging information.

NOTE *I've omitted the `print()` statements for debugging from the code listings that follow. You can find them in the full code listing and on the book's GitHub repository.*

The next global variables are for working with the audio input. `CHUNK` sets the number of data samples read at a time using `PyAudio`, and `FORMAT` specifies that the audio data will consist of 16-bit integers. You set `CHANNELS` to `1`, since you'll be using a single-channel microphone,

and `SAMPLE_RATE` to `16000` for consistency with the ML training data. `RECORD_SECONDS` indicates that you'll be grouping the audio into one-second increments (which you'll stitch together into overlapping two-second clips, as discussed earlier). You calculate the number of chunks in each one-second recording as `NCHUNKS`. You'll use `ND` and `NDH` to implement the overlapping technique—more on that later.

Finally, you initialize the device index number of the microphone to `-1`.
❶ You'll need to update this value at the command line once you know your microphone's index. Here's a function to help you figure that out. You'll be able to call this function as a command line option.

```
def list_devices():

    """list pyaudio devices"""

    # initialize pyaudio

    ❶ p = pyaudio.PyAudio()

    # get device list

    index = None

    ❷ nDevices = p.get_device_count()

    print("\naudioml.py:\nFound the following input devices:")

    ❸ for i in range(nDevices):

        deviceInfo = p.get_device_info_by_index(i)

        if deviceInfo['maxInputChannels'] > 0:

            print(deviceInfo['index'], deviceInfo['name'],

                  deviceInfo['defaultSampleRate'])
```

```
# clean up
```

```
④ p.terminate()
```

You initialize `PyAudio` ① and get a count of the audio devices it detects ②. Then you iterate through the devices ③. For each one, you retrieve information about the device using `get_device_info_by_index()` and print out devices with one or more input channels—that is, microphones. You finish by cleaning up `PyAudio` ④.

Here's what a typical output of the function looks like:

audioml.py:

Found the following input devices:

1 Mico: USB Audio (hw:3,0) 16000.0

This indicates there's an input device called Mico with a default sample rate of 16,000 and an index of 1.

Taking In Audio Data

One of the main tasks for the Pi is to continuously take in the audio input from the microphone and break it up into clips that you can run inference on. You create a `get_live_input()` function for this purpose. It takes in the `interpreter` object needed to work with the TensorFlow Lite model. Here's the start of the function:

```
def get_live_input(interpreter):
```

```
    # create a queue object
```

```
    ① dataq = Queue()
```

```
    # start inference process
```

```
❷ proc = Process(target = inference_process, args=(dataq, interpreter))

proc.start()
```

As we discussed in **“How It Works,”** you’ll need to use separate processes for reading the audio data and doing the inference to avoid missing any input. You create a `multiprocessing. Queue` object that the processes will use to communicate with each other ❶. Then you create the inference process using `multiprocessing. Process()` ❷. You specify the name of the handler function for the process as `inference_process`, which takes the `dataq` and `interpreter` objects as arguments (we’ll view this function later). You next start the process so the inference will run parallel to the data capture.

You continue the `get_live_input()` function by initializing `PyAudio`:

```
# initialize pyaudio

❶ p = pyaudio.PyAudio()

print('opening stream...')

❷ stream = p.open(format = FORMAT,

                  channels = CHANNELS,

                  rate = SAMPLE_RATE,

                  input = True,

                  frames_per_buffer = CHUNK,

                  input_device_index = dev_index)

# discard first 1 second

❸ for i in range(0, NCHUNKS):
```



```
data = stream.read(CHUNK, exception_on_overflow = False)
```

You create a `PyAudio` object `p` ❶ and open an audio input stream ❷, using some of your global variables as parameters. Then you discard the first one second of data ❸. This is to disregard any spurious data that comes in when the microphone is enabled for the first time.

Now you're ready to start reading the data:

```
# count for gathering two frames at a time
```

```
❶ count = 0
```

```
❷ inference_data = np.zeros((ND,), dtype=np.int16)
```

```
print("Listening...")
```

```
try:
```

```
❸ while True:
```

```
    chunks = []
```

```
❹ for i in range(0, NCHUNKS):
```

```
    data = stream.read(CHUNK, exception_on_overflow = False)
```

```
    chunks.append(data)
```

```
# process data
```

```
buffer = b''.join(chunks)
```

```
❺ audio_data = np.frombuffer(buffer, dtype=np.int16)
```

You initialize `count` to 0 ❶. You'll use this variable to keep track of the number of one-second frames of audio data read in. Then you initialize a 16-bit array `inference_data` with zeros ❷. It has `ND` elements, which

corresponds to two seconds of audio. You next enter a `while` loop to process the audio data continuously ❸. In it, you use a `for` loop ❹ to read in one second of audio data, one chunk at a time, appending those chunks to the list `chunks`. Once you have a full second of data, you convert it into a `numpy` array ❺.

Next, still within the `while` loop started in the previous listing, you implement the technique we discussed in [“How It Works”](#) to create overlapping two-second audio clips. You get help from your `NDH` global variable.

```
if count == 0:

    # set first half

    ❶ inference_data[:NDH] = audio_data

    count += 1

elif count == 1:

    # set second half

    ❷ inference_data[NDH:] = audio_data

    # add data to queue

    ❸ dataq.put(inference_data)

    count += 1

else:

    # move second half to first half

    ❹ inference_data[:NDH] = inference_data[NDH:]

    # set second half
```

```
⑤ inference_data[NDH:] = audio_data
```

```
# add data to queue
```

```
⑥ dataq.put(inference_data)
```

The very first time a one-second frame is read in, it's stored in the first half of `inference_data` ①. The next frame that comes in is stored in the second half of `inference_data` ②. Now you have a full two seconds of audio data, so you put `inference_data` into the queue for the inference process to pick it up ③. For every subsequent frame, the second half of the data is moved to the first half of `inference_data` ④, the new data is set to the second half ⑤, and `inference_data` is added to the queue ⑥. This creates the desired one-second overlap between each consecutive two-second audio clip.

The `while` loop occurs inside a `try` block. To exit the loop, you just need to press CTRL-C and trigger the following `except` block:

```
except KeyboardInterrupt:
```

```
    print("exiting...")
```

```
    stream.stop_stream()
```

```
    stream.close()
```

```
    p.terminate()
```

This `except` block performs some basic cleanup by stopping and closing the stream and by terminating `PyAudio`.

Preparing the Audio Data

Next, you'll create a few functions to prepare the audio data for inference. First is `process_audio_data()`, which takes in a raw two-second clip of audio data pulled from the queue and extracts the most interest-

ing one second of audio from it, based on peak amplitude. We'll look at this function across several listings:

```
def process_audio_data(waveform):  
  
    # compute peak to peak based on scaling by max 16-bit value  
  
    ❶ PTP = np.ptp(waveform / 32768.0)  
  
    # return None if too silent  
  
    ❷ if PTP < 0.3:  
  
        return []
```

You want to skip doing any inference on the microphone audio input if nobody is talking. There will always be some noise in the environment, however, so you can't simply look for the signal to be 0. Instead, you'll skip inference if the peak-to-peak amplitude (the difference between the highest value and the lowest value) of the audio is below a certain threshold. For this, you first divide the audio by 32768 to normalize it to a range of (-1, 1), and you pass the result to `np.ptp()` to get the peak-to-peak amplitude ❶. The normalization makes it easier to express the threshold as a fraction. You return an empty list (which will bypass the inference process) if the peak-to-peak amplitude is below 0.3 ❷. You may need to adjust this threshold value depending on the noise level of your environment.

The `process_audio_data()` function continues with another technique for normalizing any audio data that won't be skipped:

```
# normalize audio  
  
wabs = np.abs(waveform)  
  
wmax = np.max(wabs)
```

```
❶ waveform = waveform / wmax

# compute peak to peak based on normalized waveform

❷ PTP = np.ptp(waveform)

# scale and center

❸ waveform = 2.0*(waveform - np.min(waveform))/PTP - 1
```

When you normalized the data before skipping quiet audio samples, you divided the audio by 32,768, the maximum possible value of a 16-bit signed integer. In most cases, however, the peak amplitude of the audio data will be well below this value. Now you want to normalize the audio such that its maximum amplitude, whatever that may be, is scaled to 1. To do this, you first determine the peak amplitude in the audio signal and then divide the signal by that amplitude value ❶. Then you compute the new peak-to-peak value of the normalized audio ❷ and use this value to scale and center the data ❸. Specifically, the expression `(waveform - np.min(waveform))/PTP` will scale the waveform values to the range (0, 1). Multiplying this by 2 and subtracting 1 will put the values in the range (-1, 1), which is what you need.

The next part of the function extracts one second of audio from the data:

```
# extract 16000 len (1 second) of data

❶ max_index = np.argmax(waveform)

❷ start_index = max(0, max_index-8000)

❸ end_index = min(max_index+8000, waveform.shape[0])

❹ waveform = waveform[start_index:end_index]

# padding for files with less than 16000 samples
```

```
waveform_padded = np.zeros((16000,))

waveform_padded[:waveform.shape[0]] = waveform

return waveform_padded
```

You want to make sure you're getting the most interesting one second of the data, so you find the array index where the audio amplitude is at the maximum ❶. Then you try to grab 8,000 values before ❷ and after ❸ this index to get a full second of data, using `max()` and `min()` to ensure that the start and end indices don't fall out of range of the original clip. You use slicing to extract the relevant audio data ❹. Because of the `max()` and `min()` operations, you may end up with less than 16,000 samples, but the neural network strictly requires each input to be 16,000 samples long. To address this problem, you pad the data with zeros, using the same `numpy` techniques you saw during training. Then you return the result.

Figure 15-7 summarizes the `process_audio_data()` function by showing an example waveform at the various stages of processing.

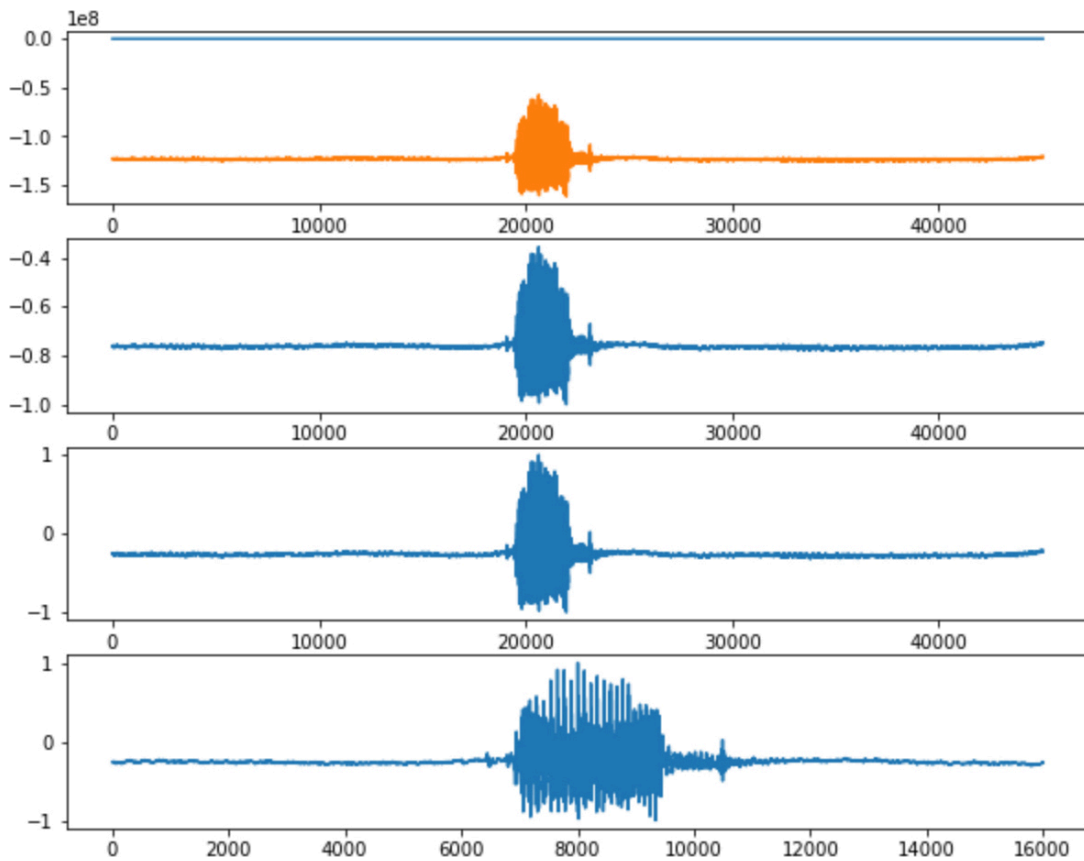


Figure 15-7: The audio preparation process at various stages

The top waveform in [Figure 15-7](#) shows the unprocessed audio. The second waveform shows the audio with the values normalized to range $(-1, 1)$. The third waveform shows the audio after a shift and scale—notice on the y-axis how the waveform now fills the entire $(-1, 1)$ range. The fourth waveform consists of 16,000 samples extracted from the third one, centered on the peak amplitude.

Next, you need a `get_spectrogram()` function for computing the spectrogram of the audio data:

```
def get_spectrogram(waveform):
```

```
    ❶ waveform_padded = process_audio_data(waveform)
```

```
    ❷ if not len(waveform_padded):
```

```
        return []
```

```
# compute spectrogram
```

```
❸ f, t, Zxx = signal.stft(waveform_padded, fs=16000, nperseg=255,
```

```
noverlap = 124, nfft=256)
```

```
# output is complex, so take abs value
```

```
❹ spectrogram = np.abs(Zxx)
```

```
return spectrogram
```

You call your `process_audio_data()` function to prepare the audio ❶. If the function returns an empty list (because the audio is too quiet), `get_spectrogram()` returns an empty list as well ❷. Next, you compute the spectrogram with `signal.stft()` from `scipy`, exactly as you did when training the model ❸. You then calculate the absolute value of the STFT ❹ to convert from complex numbers—again, as you did during training—and return the result.

Running Inference

The heart of this project is using your trained model to run inference on the incoming audio data and identify any spoken commands. Recall that this occurs in a separate process from the code for taking in audio data from the microphone. Here's the handler function that coordinates this process:

```
def inference_process(dataq, interpreter):
```

```
    success = False
```

```
    while True:
```

```
        ❶ if not dataq.empty():
```

```
            # get data from queue
```



```
❷ inference_data = dataq.get()

# run inference only if previous one was not successful

❸ if not success:

    success = run_inference(inference_data, interpreter)

else:

    # skipping, reset flag for next time

❹ success = False
```

The inference process runs continuously inside a `while`. Within this loop, you check if there's any data in the queue ❶, and if so, you retrieve it ❷. Then you run inference on it with the `run_inference()` function, which we'll look at next, but only if the `success` flag is `False` ❸. This flag keeps you from responding to the same speech command twice. Recall that because of the overlap technique, the second half of one audio clip will be repeated as the first half of the next clip. This lets you catch any audio commands that might be split across two frames, but it means that once you have a successful inference, you should skip the next element in the queue because it will have a portion of the audio from the previous element. When you do a skip like this, you reset `success` to `False` ❹ to start running inference again on the next piece of data that comes in.

Now let's look at the `run_inference()` function, where the inference is actually carried out:

```
def run_inference(waveform, interpreter):

    # get spectrogram data

    ❶ spectrogram = get_spectrogram(waveform)
```

```

if not len(spectrogram):

    return False

# get input and output tensors details

❷ input_details = interpreter.get_input_details()

❸ output_details = interpreter.get_output_details()

```

The function takes in the raw audio data (`waveform`) for interacting with your TensorFlow Lite model (`interpreter`). You call `get_spectrogram()` to process the audio and generate the spectrogram ❶, and if the audio was too quiet, you return `False`. Then you get the input ❷ and output ❸ details from the TensorFlow Lite interpreter. These tell you what the model is expecting as input and what you can expect from it as output. This is what `input_details` looks like:

```

[{'name': 'serving_default_input_5:0', 'index': 0, 'shape': array([1, 129,
124, 1]),

  'shape_signature': array([-1, 129, 124, 1]), 'dtype': <class
'numpy.float32'>,

  'quantization': (0.0, 0), 'quantization_parameters': {'scales': array([],
dtype=float32),

  'zero_points': array([], dtype=int32), 'quantized_dimension': 0},
'sparsity_parameters': {}}]

```

Notice that `input_details` is an array with a dictionary inside it. The `'shape'` entry is especially of interest: `array([1, 129, 124, 1])`. You've already ensured that your spectrogram, which will be the input to the interpreter, is shaped to this value. The `'index'` entry is just the index of the tensor in the tensor list inside the interpreter, and `'dtype'` is the expected data type of the input, which in this case is `float32`, a

signed 32-bit float. You'll need to reference both `'index'` and `'dtype'` later in the `run_inference()` function.

Here's `output_details`:

```
[{'name': 'StatefulPartitionedCall:0', 'index': 17, 'shape': array([1, 8]),
'shape_signature':

array([-1, 8]), 'dtype': <class 'numpy.float32'>, 'quantization': (0.0, 0),

'quantization_parameters': {'scales': array([], dtype=float32),
'zero_points':

array([], dtype=int32), 'quantized_dimension': 0}, 'sparsity_parameters':
{}}]
```

Notice the `'shape'` entry in this dictionary. It shows that the output will be an array of shape (1, 8). The shape corresponds to the label IDs of the eight speech commands.

You continue the `run_inference()` function by actually running inference on the input data:

```
# set input
```

```
❶ input_data = spectrogram.astype(np.float32)
```

```
❷ interpreter.set_tensor(input_details[0]['index'], input_data)
```

```
# run interpreter
```

```
print("running inference...")
```

```
❸ interpreter.invoke()
```

```
# get output
```

```
❹ output_data = interpreter.get_tensor(output_details[0]['index'])
```

```
⑤ yvals = output_data[0]
```

```
print(yvals)
```

First you convert the spectrogram data to 32-bit floating point values ❶. Recall that your audio data started as 16-bit integers. The scaling and other processing operations converted the data to 64-bit floats, but as you saw in `input_details`, the TensorFlow Lite model requires 32-bit floats, which is the reason for the conversion. You next set the input value to the appropriate tensor inside the interpreter ❷. Here the `[0]` accesses the first (and only) element in `input_details`, which as you saw is a dictionary, and `['index']` retrieves the value under that key in the dictionary to specify which tensor you're setting. You run inference on the input using the `invoke()` method ❸. Then you retrieve the output tensor using similar indexing to the input ❹ and get the output itself by extracting the first element from the `output_data` array ❺. (Since you provided only one input, you expect only one output.) Here's an example of what `yvals` looks like:

```
[ 6.640185 -26.032831 -26.028618  8.746256  62.545185 -0.5698182 -15.045679 -29.140179 ]
```

These eight numbers correspond to the eight commands you trained the model with. The values indicate the likelihood of the input data being each word. In this particular array, the value at index 4 is by far the largest, so that's what the neural network is predicting as the most probable answer. Here's how you interpret the result:

```
# Important! This should exactly match training labels/ids.
```

```
commands = ['up', 'no', 'stop', 'left', 'right', 'go', 'down', 'yes']
```

```
print(">>> " + commands[np.argmax(output_data[0]).upper()])
```

You define a `commands` list in the same order as you used during training. It's important to keep the order consistent across training and inference, or you'll end up misinterpreting the results! Then you use

`np.argmax()` to get the index of the highest value in the output data and use that index to pick up the corresponding string from `commands`.

Writing the `main()` Function

Now let's look at the `main()` function, which brings everything together:

```
def main():
```

```
    # globals set in this function
```

```
    global VERBOSE_DEBUG
```

```
    # create parser
```

```
    descStr = "This program does ML inference on audio data."
```

```
    parser = argparse.ArgumentParser(description=descStr)
```

```
    # add a mutually exclusive group
```

```
    ❶ group = parser.add_mutually_exclusive_group(required=True)
```

```
    # add mutually exclusive arguments
```

```
    ❷ group.add_argument('--list', action='store_true', required=False)
```

```
    ❸ group.add_argument('--input', dest='wavfile_name', required=False)
```

```
    ❹ group.add_argument('--index', dest='index', required=False)
```

```
    # add other arguments
```

```
    ❺ parser.add_argument('--verbose', action='store_true', required=False)
```

```
    # parse args
```

```
args = parser.parse_args()
```

You start by setting `VERBOSE_DEBUG` as a global, since you'll be setting it in this function and don't want it to be treated as a local variable. Then you create a familiar `argparse.ArgumentParser` object and add a mutually exclusive group to the parser ❶, since some of your command line options won't be compatible with each other. Those are the `--list` option ❷, which will list all the `PyAudio` devices so you can get your microphone's index number; the `--input` option ❸, which lets you specify a WAV file to use as input instead of live data from the microphone (useful for testing); and the `--index` option ❹, which starts capturing audio and running inference using the microphone with the specified index. You also add the non-mutually exclusive `--verbose` option ❺ to print out detailed debug information as the program is run.

Next, you create the TensorFlow Lite interpreter so you can use the ML model:

```
# load TF Lite model

interpreter = Interpreter('audioml.tflite')

interpreter.allocate_tensors()
```

Here you create an `Interpreter` object, passing it the `audioml.tflite` file with the model you created during training. Then you call `allocate_tensors()` to prepare the necessary tensors for running the inference.

The `main()` function finishes with branches for the different command line arguments:

```
# check verbose flag

if args.verbose:

    VERBOSE_DEBUG = True
```

```
# test WAV file

if args.wavfile_name:

    ❶ wavfile_name = args.wavfile_name

    # get audio data

    ❷ rate, waveform = wavfile.read(wavfile_name)

    # run inference

    ❸ run_inference(waveform, interpreter)

elif args.list:

    # list devices

    ❹ list_devices()

else:

    # store device index

    ❺ dev_index = int(args.index)

    # get live audio

    ❻ get_live_input(interpreter)

print("done.")
```

If the `--input` command line option is used, you get the name of the WAV file ❶ and read its contents ❷. The resulting data is passed along for inference ❸. If the `--list` option is used, you call your `list_devices()` function ❹. If the `--index` option is used, you parse the device index ❺ and start processing live audio by calling the `get_live_input()` function ❻.

Running the Speech Recognition System

To run the project, gather your Python code and the *audioml.tflite* file into a folder on your Pi. For testing, you can also download the *right.wav* file from the book's GitHub repository and add that to the folder. You can work with your Pi via SSH, as explained in [Appendix B](#).

First, try using the `--input` command line option to run inference on a WAV file:

```
$ sudo python audioml.py --input right.wav
```

Here's the output:

running inference...

```
[ 6.640185 -26.032831 -26.028618  8.746256  62.545185 -0.5698182  
-15.045679 -29.140179 ]
```

❶ >>> RIGHT

run_inference: 0.029174549999879673s

done.

Notice that the program has correctly identified the *right* command recorded on the WAV file ❶.

Now plug your microphone into the Pi and use the `--list` option to determine its index number, as shown here:

```
$ sudo python audioml.py --list
```

Your output should be similar to the following:

audioml.py:

Found the following input devices:

1 Mico: USB Audio (hw:3,0) 16000.0

done.

In this example, the microphone has index `1`. Use that number to run the `--index` command to do some live speech detection! Here's an example run:

```
$ sudo python audioml.py --index 1
```

```
-- snip --
```

opening stream...

Listening...

running inference...

```
[-2.647918  0.17592785 -3.3615346  6.6812882  4.472283  -3.7535028  
1.2349942  1.8546474 ]
```

❶ >>> LEFT

run_inference: 0.03520956500142347s

running inference...

```
[-2.7683923 -5.9614644 -8.532391  6.906795  19.197264  -4.0255833  
1.7236844 -4.374415 ]
```

❷ >>> RIGHT

run_inference: 0.03026762299850816s

```
-- snip --
```

```
^C
```

```
KeyboardInterrupt
```

```
exiting...
```

```
done.
```

After starting the program and getting the “Listening . . .” prompt, I spoke the words *left* and *right*. The output at ❶ and ❷ indicates that the program was able to identify the commands correctly.

Try running the program with the `--verbose` option to see more information about how it’s working. Also, try speaking different commands in quick succession to verify whether the multiprocessing and overlapping techniques are working.

Summary

This chapter introduced you to the world of machine learning. You learned how to train a deep neural network to recognize speech commands using the TensorFlow framework, and you converted the resulting model to a TensorFlow Lite format for use on a resource-constrained Raspberry Pi. You also learned about spectrograms and the importance of processing input data before ML training. You practiced using Python multiprocessing, reading USB microphone input on a Raspberry Pi using `PyAudio`, and running a TensorFlow Lite interpreter for ML inference.

Experiments!

1. Now that you know how to process speech commands on a Raspberry Pi, you can build an assistive device that responds to those commands by doing more than printing out the identified words. For example, you could use the commands *left*, *right*, *up*, *down*, *stop*, and *go* to con-

trol a camera (or laser!) mounted on a pan/tilt mount. Hint: you'll need to retrain the ML model with just these six commands. You'll also need to get a two-axis pan/tilt bracket with two servo motors attached. The servos will be connected to the Raspberry Pi and controlled based on the inference results.

2. Read about the *mel spectrogram*, a variant of the spectrogram you used for this project that's better suited for human speech data.
3. Try modifying the neural network by adding or removing some layers. For example, remove the second Conv2D layer. See how the changes affect the training accuracy of the model and the inference accuracy on the Pi.
4. This project used an ad hoc neural network, but there are also pre-trained neural networks available that you could leverage. For example, read up on MobileNet V2. What changes are needed to adapt your project to use this network instead?

The Complete Code

Here's a complete listing of the code that goes on the Raspberry Pi, including the `print()` statements for verbose debugging. The Google Colab notebook code can be found at

<https://github.com/mkvenkit/pp2e/blob/main/audioml/audioml.ipynb>.

"""

simple_audio.py

This programs collects audio data from an I2S mic on the Raspberry Pi and runs the TensorFlow Lite interpreter on a per-build model.

Author: Mahesh Venkitachalam

"""

from scipy.io import wavfile

```
from scipy import signal

import numpy as np

import argparse

import pyaudio

import wave

import time

from tfliite_runtime.interpreter import Interpreter

from multiprocessing import Process, Queue

VERBOSE_DEBUG = False

CHUNK = 4000          # choose a value divisible by SAMPLE_RATE

FORMAT = pyaudio.paInt16

CHANNELS = 1

SAMPLE_RATE = 16000

RECORD_SECONDS = 1

NCHUNKS = int((SAMPLE_RATE * RECORD_SECONDS) / CHUNK)

ND = 2 * SAMPLE_RATE * RECORD_SECONDS

NDH = ND // 2

# device index of microphone

dev_index = -1
```

```
def list_devices():

    """list pyaudio devices"""

    # initialize pyaudio

    p = pyaudio.PyAudio()

    # get device list

    index = None

    nDevices = p.get_device_count()

    print("\naudioml.py:\nFound the following input devices:")

    for i in range(nDevices):

        deviceInfo = p.get_device_info_by_index(i)

        if deviceInfo['maxInputChannels'] > 0:

            print(deviceInfo['index'], deviceInfo['name'],
deviceInfo['defaultSampleRate'])

    # clean up

    p.terminate()

def inference_process(dataq, interpreter):

    """infererence process handler"""

    success = False

    while True:

        if not dataq.empty():
```

```
# get data from queue

inference_data = dataq.get()

# run inference only if previous one was not success

# otherwise we will get duplicate results because of

# overlap in input data

if not success:

    success = run_inference(inference_data, interpreter)

else:

    # skipping, reset flag for next time

    success = False

def process_audio_data(waveform):

    """Process audio input.

    This function takes in raw audio data from a WAV file and does scaling

    and padding to 16000 length.

    """

    if VERBOSE_DEBUG:

        print("waveform:", waveform.shape, waveform.dtype,

              type(waveform))

        print(waveform[:5])

    # compute peak to peak based on scaling by max 16-bit value
```

```
PTP = np.ptp(waveform / 32768.0)

if VERBOSE_DEBUG:

    print("peak-to-peak (16 bit scaling): {}".format(PTP))

# return None if too silent

if PTP < 0.3:

    return []

# normalize audio

wabs = np.abs(waveform)

wmax = np.max(wabs)

waveform = waveform / wmax

# compute peak to peak based on normalized waveform

PTP = np.ptp(waveform)

if VERBOSE_DEBUG:

    print("peak-to-peak (after normalize): {}".format(PTP))

    print("After normalization:")

    print("waveform:", waveform.shape, waveform.dtype,
type(waveform))

    print(waveform[:5])

# scale and center

waveform = 2.0*(waveform - np.min(waveform))/PTP - 1
```

```
# extract 16000 len (1 second) of data

max_index = np.argmax(waveform)

start_index = max(0, max_index-8000)

end_index = min(max_index+8000, waveform.shape[0])

waveform = waveform[start_index:end_index]

# padding for files with less than 16000 samples

if VERBOSE_DEBUG:

    print("After padding:")

    waveform_padded = np.zeros((16000,))

    waveform_padded[:waveform.shape[0]] = waveform

    if VERBOSE_DEBUG:

        print("waveform_padded:", waveform_padded.shape,

              waveform_padded.dtype, type(waveform_padded))

        print(waveform_padded[:5])

    return waveform_padded

def get_spectrogram(waveform):

    """computes spectrogram from audio data"""

    waveform_padded = process_audio_data(waveform)

    if not len(waveform_padded):
```



```
    return []

# compute spectrogram

f, t, Zxx = signal.stft(waveform_padded, fs=16000, nperseg=255,

    noverlap = 124, nfft=256)

# output is complex, so take abs value

spectrogram = np.abs(Zxx)

if VERBOSE_DEBUG:

    print("spectrogram:", spectrogram.shape, type(spectrogram))

    print(spectrogram[0, 0])

return spectrogram

def run_inference(waveform, interpreter):

    # start timing

    start = time.perf_counter()

    # get spectrogram data

    spectrogram = get_spectrogram(waveform)

    if not len(spectrogram):

        if VERBOSE_DEBUG:

            print("Too silent. Skipping...")

        return False
```

```
if VERBOSE_DEBUG:

    print("spectrogram: %s, %s, %s" % (type(spectrogram),

        spectrogram.dtype, spectrogram.shape))

# get input and output tensors details

input_details = interpreter.get_input_details()

output_details = interpreter.get_output_details()

if VERBOSE_DEBUG:

    print("input_details: {}".format(input_details))

    print("output_details: {}".format(output_details))

# reshape spectrogram to match interpreter requirement

spectrogram = np.reshape(spectrogram, (-1, spectrogram.shape[0],

        spectrogram.shape[1], 1))

# set input

input_data = spectrogram.astype(np.float32)

interpreter.set_tensor(input_details[0]['index'], input_data)

# run interpreter

print("running inference...")

interpreter.invoke()

# get output
```

```
output_data = interpreter.get_tensor(output_details[0]['index'])

yvals = output_data[0]

if VERBOSE_DEBUG:

    print(output_data)

print(yvals)

# Important! This should exactly match training labels/ids.

commands = ['up', 'no', 'stop', 'left', 'right', 'go', 'down', 'yes']

print(">>> " + commands[np.argmax(output_data[0]).upper()])

# stop timing

end = time.perf_counter()

print("run_inference: {}s".format(end - start))

# return success

return True

def get_live_input(interpreter):

    """this function gets live input from the microphone

    and runs inference on it"""

    # create a queue object

    dataq = Queue()

    # start inference process
```

```
proc = Process(target = inference_process, args=(dataq, interpreter))

proc.start()

# initialize pyaudio

p = pyaudio.PyAudio()

print('opening stream...')

stream = p.open(format = FORMAT,

                 channels = CHANNELS,

                 rate = SAMPLE_RATE,

                 input = True,

                 frames_per_buffer = CHUNK,

                 input_device_index = dev_index)

# discard first 1 second

for i in range(0, NCHUNKS):

    data = stream.read(CHUNK, exception_on_overflow = False)

# count for gathering two frames at a time

count = 0

inference_data = np.zeros((ND,), dtype=np.int16)

print("Listening...")

try:
```

```
while True:
```

```
    # print("Listening...")
```

```
    chunks = []
```

```
    for i in range(0, NCHUNKS):
```

```
        data = stream.read(CHUNK, exception_on_overflow = False)
```

```
        chunks.append(data)
```

```
    # process data
```

```
    buffer = b''.join(chunks)
```

```
    audio_data = np.frombuffer(buffer, dtype=np.int16)
```

```
    if count == 0:
```

```
        # set first half
```

```
        inference_data[:NDH] = audio_data
```

```
        count += 1
```

```
    elif count == 1:
```

```
        # set second half
```

```
        inference_data[NDH:] = audio_data
```

```
        # add data to queue
```

```
        dataq.put(inference_data)
```

```
        count += 1
```

```
else:

    # move second half to first half

    inference_data[:NDH] = inference_data[NDH:]

    # set second half

    inference_data[NDH:] = audio_data

    # add data to queue

    dataq.put(inference_data)

    # print("queue: {}".format(dataq.qsize()))

except KeyboardInterrupt:

    print("exiting...")

    stream.stop_stream()

    stream.close()

    p.terminate()

def main():

    """main function for the program"""

    # globals set in this function

    global VERBOSE_DEBUG

    # create parser

    descStr = "This program does ML inference on audio data."
```

```
parser = argparse.ArgumentParser(description=descStr)

# add a mutually exclusive group

group = parser.add_mutually_exclusive_group(required=True)

# add mutually exclusive arguments

group.add_argument('--list', action='store_true', required=False)

group.add_argument('--input', dest='wavfile_name', required=False)

group.add_argument('--index', dest='index', required=False)

# add other arguments

parser.add_argument('--verbose', action='store_true', required=False)

# parse args

args = parser.parse_args()

# load TF Lite model

interpreter = Interpreter('audioml.tflite')

interpreter.allocate_tensors()

# check verbose flag

if args.verbose:

    VERBOSE_DEBUG = True

# test WAV file

if args.wavfile_name:
```

```
wavfile_name = args.wavfile_name

# get audio data

rate, waveform = wavfile.read(wavfile_name)

# run inference

run_inference(waveform, interpreter)

elif args.list:

    # list devices

    list_devices()

else:

    # store device index

    dev_index = int(args.index)

    # get live audio

    get_live_input(interpreter)

    print("done.")

# main method

if __name__ == '__main__':

    main()
```
